A rapid overview of Lean

Proof compiler

Definitions, theorems, proofs

↓

consistency & correctness, guaranteed

Built on type theory

Lean knows the **type** of everything, and checks that things have the type expected.

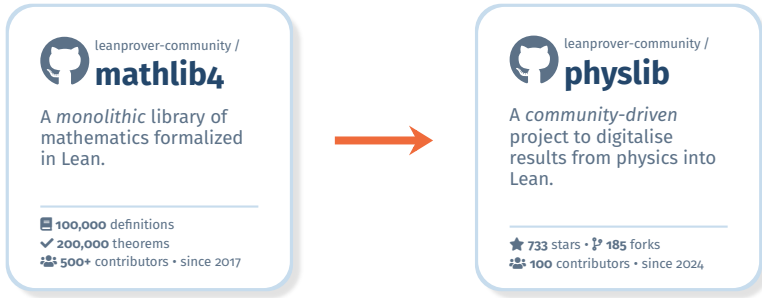
$$\underbrace{p}_{\text{proof}} : \underbrace{P}_{\text{theorem}} \qquad \underbrace{7}_{\text{term}} : \underbrace{\mathbb{N}}_{\text{type}}$$

A definition, a theorem, a proof

```
def oddSum (n : Nat) : Nat :=
  (Finset.range n).sum
    (fun i => 2 * i + 1)

theorem oddSum_eq (n : Nat) :
    oddSum n = n ^ 2 := by
  induction n with
  | zero => simp [oddSum]
  | succ n ih =>
    unfold oddSum at *
    rw [Finset.sum_range_succ, ih]
    ring
```

What I work on • Joseph Tooby-Smith



- Physlib is being heavily maintained and developed at Bath.
- Explore the consequences of the use of AI in formalization in mathematics and physics through perspectives, online forums, and in-person workshops.
- Explore the use of frontier and local models in solving conjectures related to physics, such as the Thomson problem for $N = 8$...

The Thomson problem

Conjecture

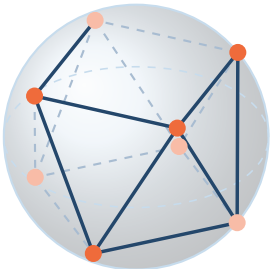
The configuration of $N = 8$ unit charges on the sphere minimising the Coulomb energy $E = \sum_{i<j} |x_i - x_j|^{-1}$ is the **square antiprism**.

Solved

Solved using AI with no human input, in the space of a couple of days (concurrently by another group as well).

Discussion that followed

Led to the Zulip discussion **Proving an open conjecture with AI + Lean**: what to do when you prove an open conjecture with minimal input.



The square antiprism, $N = 8$